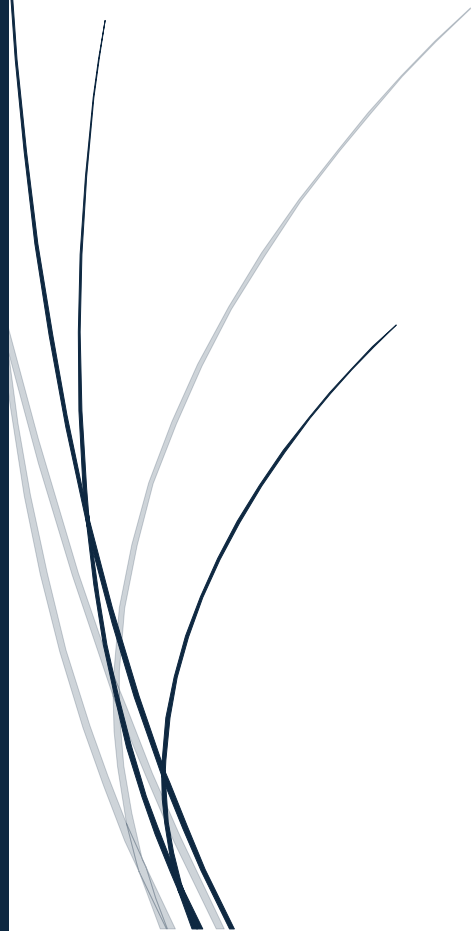
10-6-2026

Portfolio Deep Learning

Max ter Steege



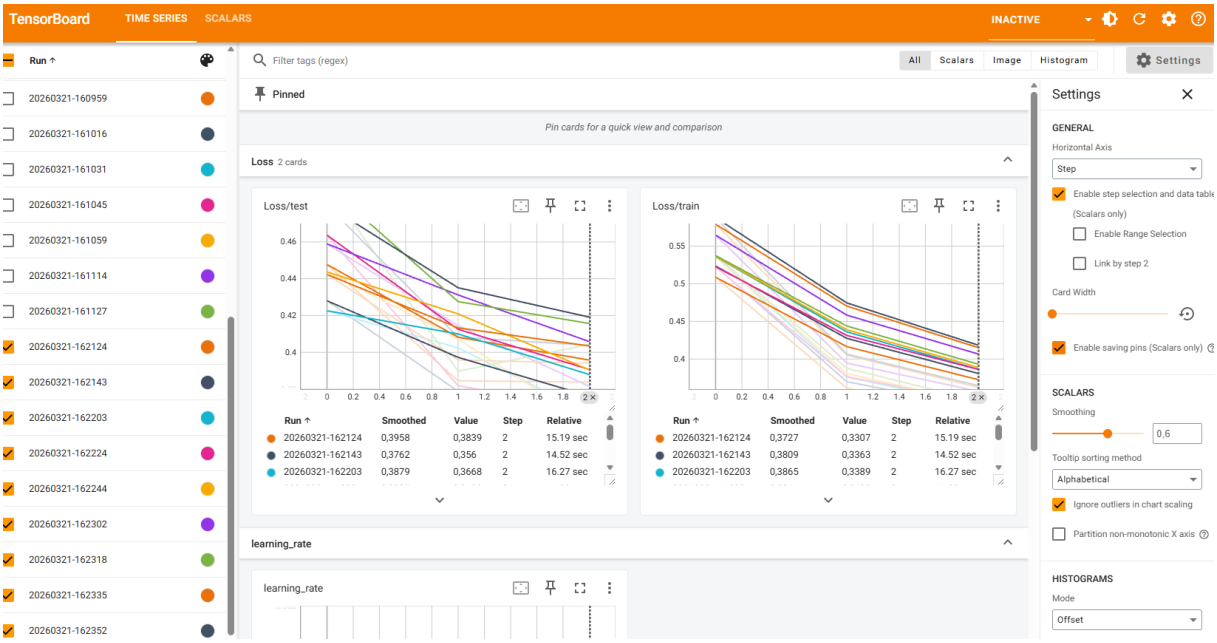
Inhoudsopgave

Portfolio assignment – College 1 – NN

Before experimenting with the hyperparameters, I have taken a screenshot of the base run with the standard parameters from the example script. See screenshot below.

Figure 1

Overview experiment hyperparameters NN in TensorBoard



There are 9 runs visible, all with a steady decline in training loss. The decline in test loss is varying a bit more, although all of them seem to be sloping downwards, hence indicating improvement in all of the runs. In the baseline experiment, the smoothed version of the following runs was the best (both for training and test):

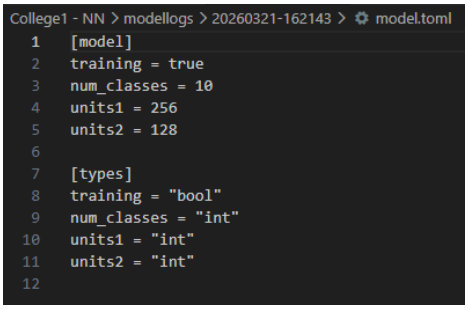
Figure 2

Best run of baseline experiment

The 162143 run has the following connected model settings:

Figure 3

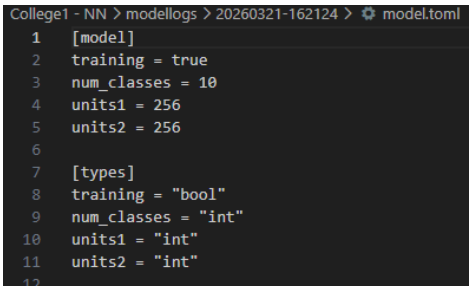
Settings 162143 run



And the 162124 run has the following:

Figure 4

Settings 162124 run



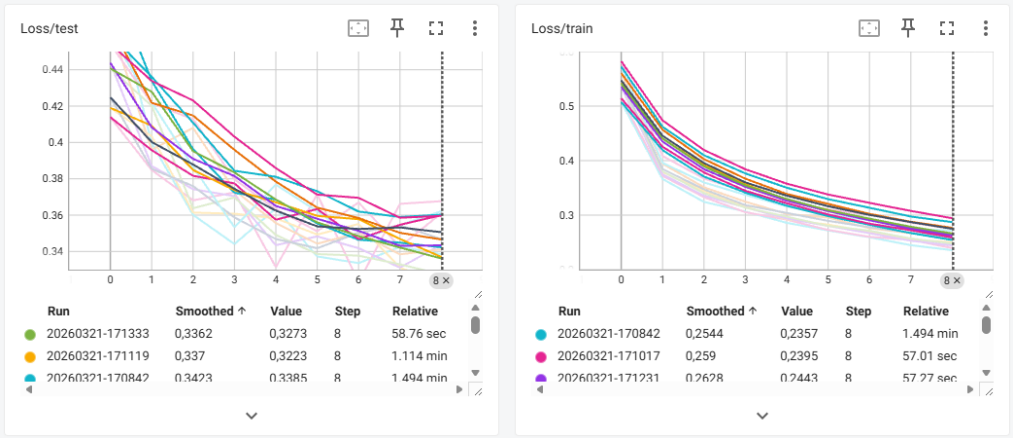
My hypothesis is as follows:

I expect the training and test loss to improve when I increase the amount of epochs and increase the amount of units. I will not increase the amount of layers (yet) since I don't want to risk overfitting. I think a lot of improvement is already possible through increasing the epochs and the unit size. Also, especially since both training and test loss lines are declining, it looks like the model is still busy learning. So I will start with increasing only the epochs to evaluate the influence.

In my first test, I increased the amount of epochs to 9. The training obviously took longer due to increasing the epochs, but the results also increased a lot. What is mainly interesting to see is that my lines are becoming more flat, so it looks like increasing the epochs took care of the fact that the model was not done with its steep learning yet. That's resolved now!

Figure 5

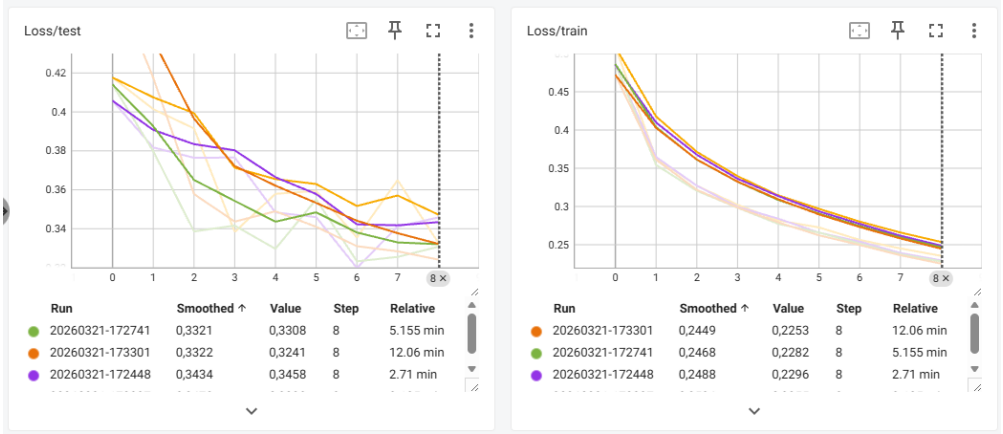
Increasing the amount of epochs, results in TensorBoard



For the final experiment, I will increase the amount of units and keep the amount of epochs at 9. Again, training took way longer due to adding 1024 in units. It actually took so long that I decided to remove 64 and 128 and keep it at 256 and 1024 (since 256 seemed to be better than the other units in previous runs). So I kept epochs at 9 and added 1024 to increase model performance and deleted the other units to improve training time a bit. The screenshot below shows that training and test loss both improved when compared to previous run.

Figure 6

Increasing the amount of units, results in TensorBoard

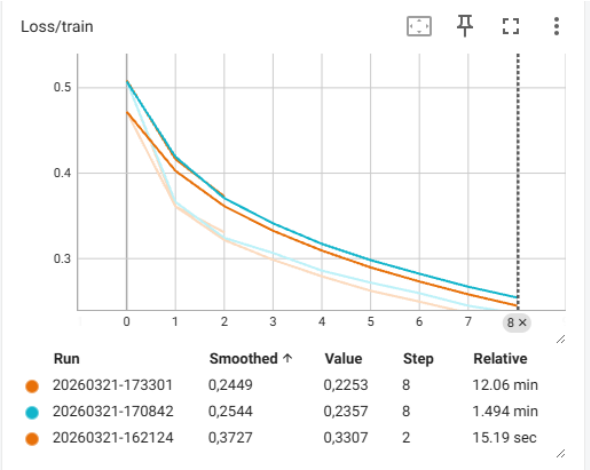


For comparison, I have added below the graphs of the training and test loss of the best models of each adjustment including the baseline, so that it's clearly visible how changing the parameters helped improve model performance.

First, the training loss. The training loss seems to improve with every iteration change. Although the last iteration change is probably not worth it. Training time went up by almost a factor of 12 while training loss only slightly improved.

Figure 7

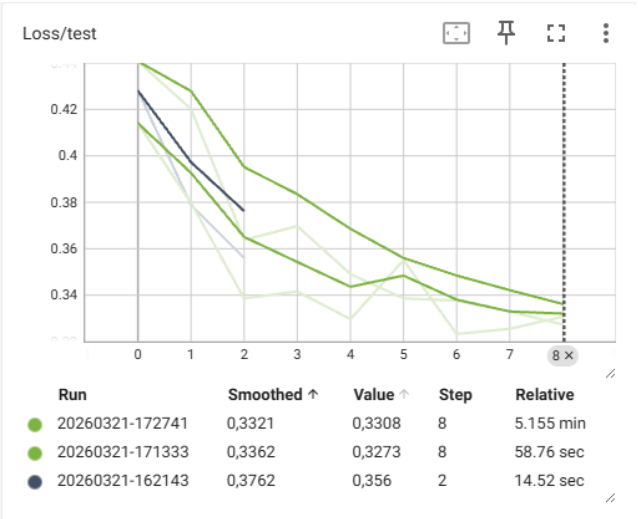
Training loss overview, results in TensorBoard



Lastly, the test loss. Although the test loss improved, the change in model performance is relatively low compared to the extra time it took to train the model. From this perspective, it's probably not worth it to train the model 5 times longer for such a small improvement. A potential gain is better found in tuning other parameters (but of course this depends on the context of the situation at hand).

Figure 8

Test loss overview, results in TensorBoard

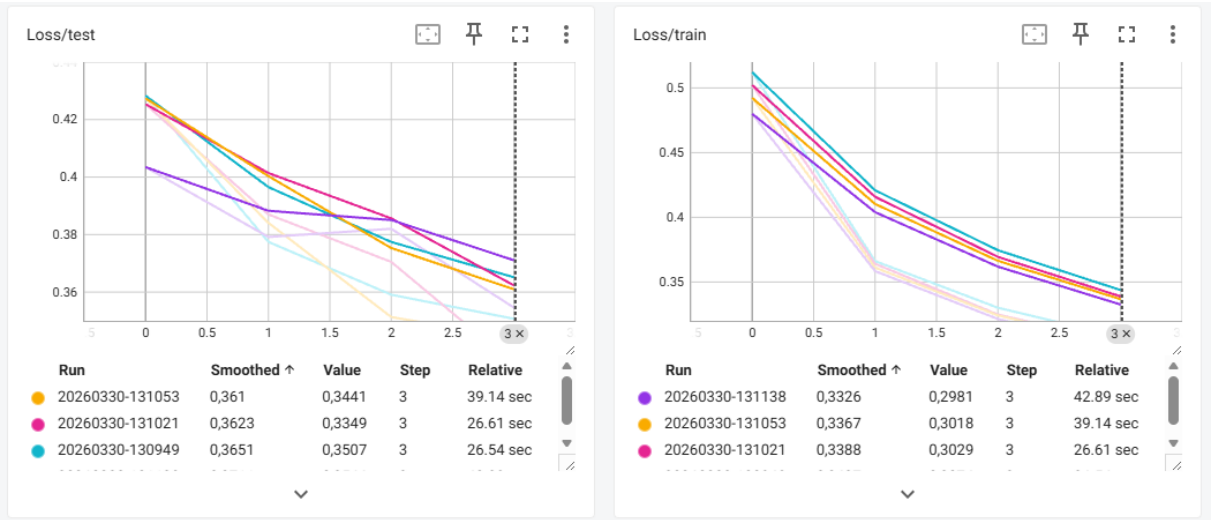


Portfolio assignment – College 2 – CNN

Since I will be applying some more changes in my model, I will start with a baseline run that is a bit faster than my last script. So I adjusted units to 256 and 512 and amount of epochs to 4.

Figure 9

Baseline run, results in TensorBoard



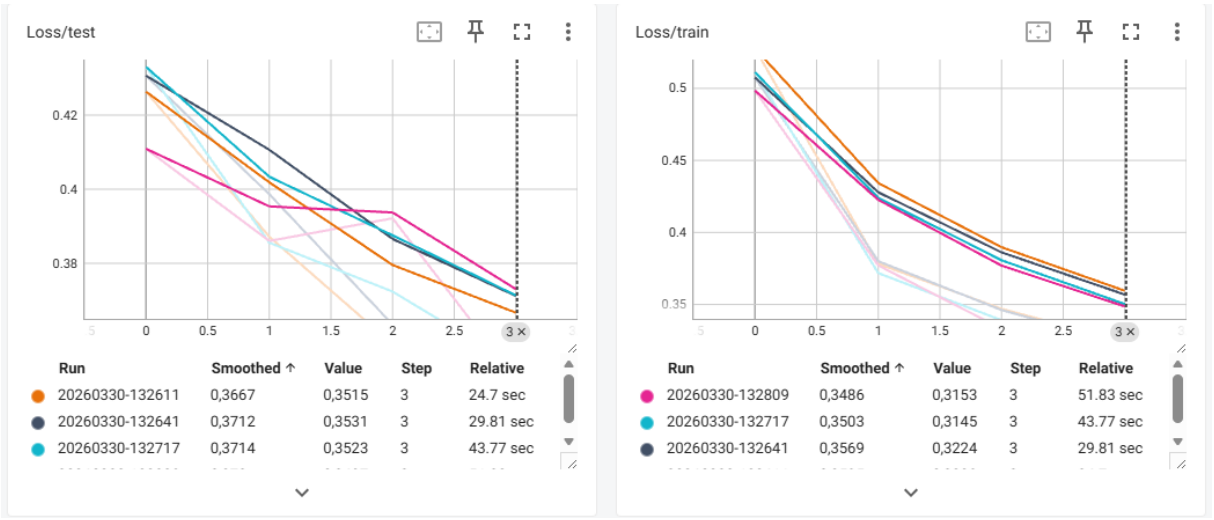
I will start with a hypothesis again.

After adding dropout, normalization, extra convolution layer and extra pooling layer, I expect the training loss and test loss to decrease, while I expect the metric value to improve. At the very least, I expect the generalization of the model to improve (especially after adding dropout).

My next run included dropout added of 0.1 after 2 layers. Results actually have gotten a bit worse, but I can imagine this happening since we are leaving out more information, hence the model will have a bit of a harder time to capture certain patterns (especially since I lowered the amount of epochs).

Figure 10

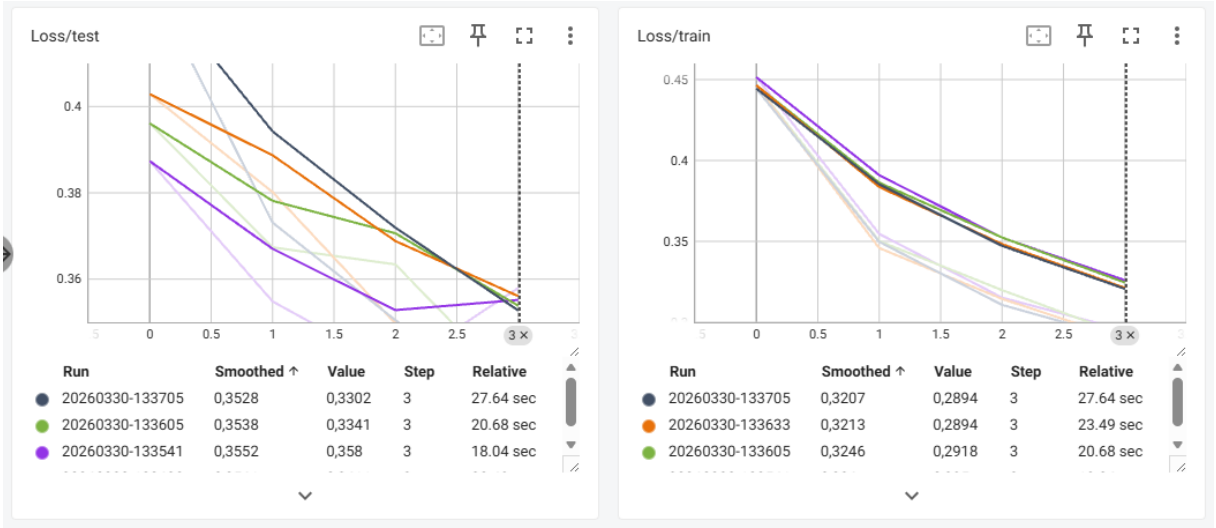
Adding dropout, results in TensorBoard



After including dropout, I added normalization and compared the results again. While still including dropout. So now we can see the results are actually improving beyond the baseline run.

Figure 11

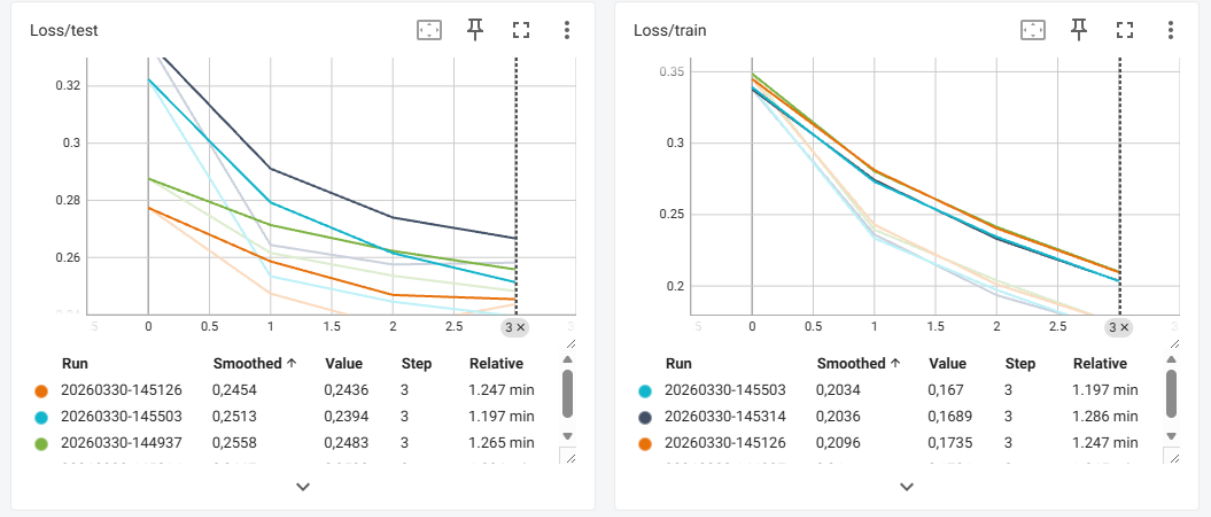
Adding normalization, results in TensorBoard



Now, convolutional and pooling added, which delivered an incredible improvement on both the training and test loss.

Figure 12

Adding convolutional and pooling, results in TensorBoard



I did all of this using Tensor Board, but now I also added logic to log via MLFLOW and reran the latest run (including convolutional and pooling), so I can show that logging works via MLFLOW.

Figure 13

Overview of logging in MLFlow

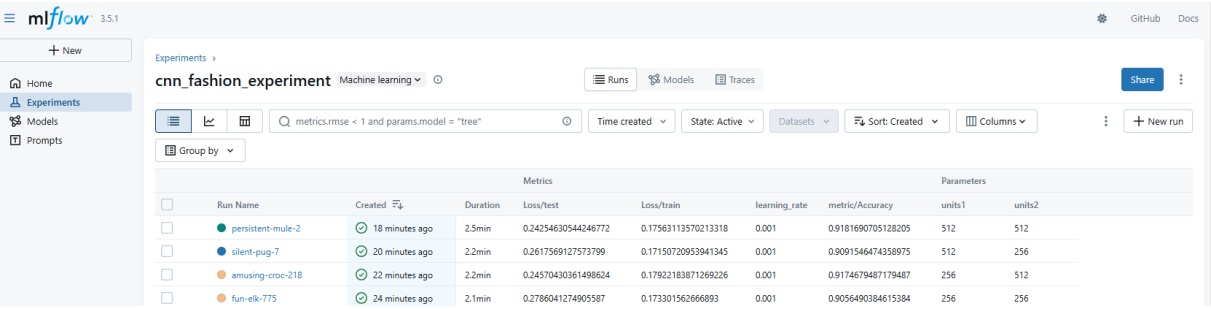
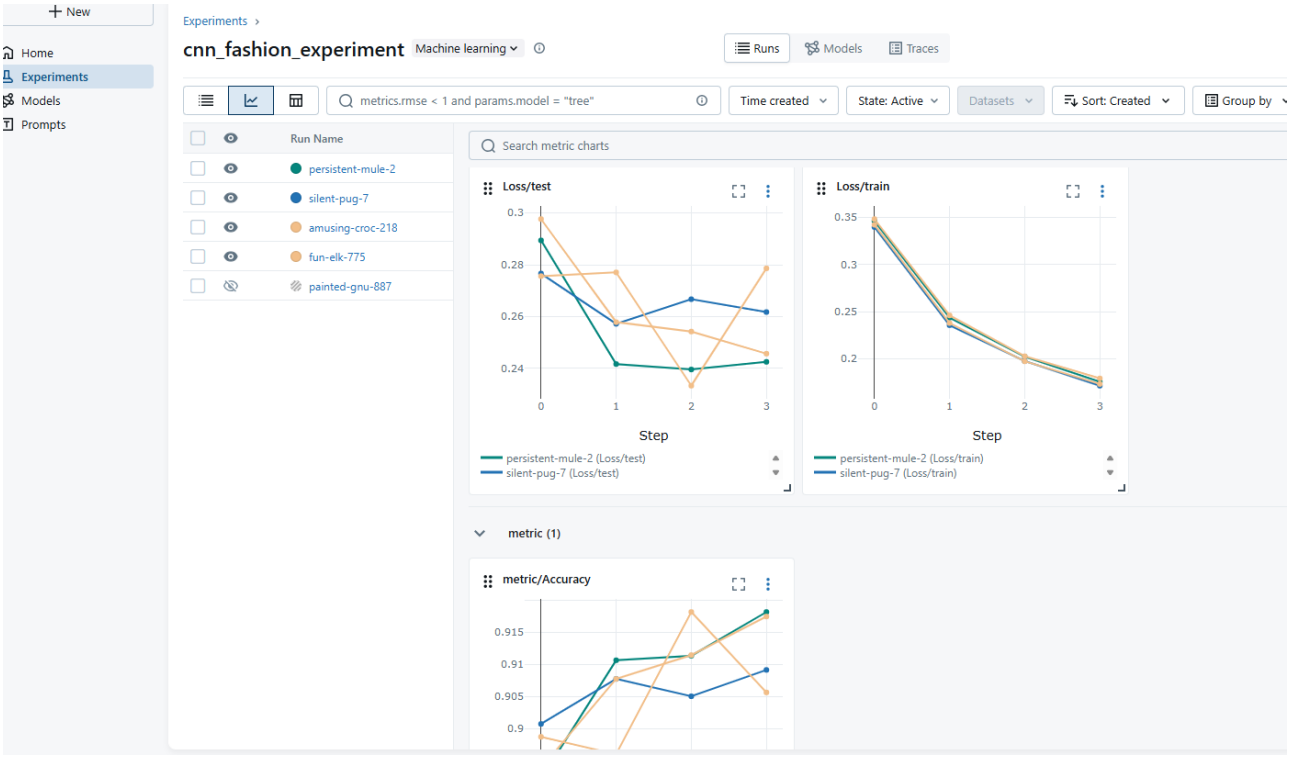


Figure 14

Overview of experiment results in MLFlow



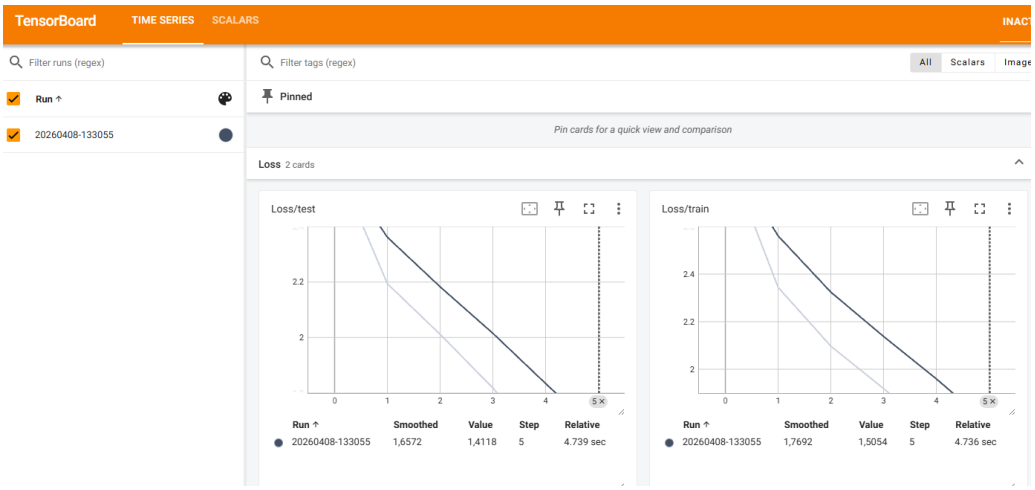
Portfolio assignment – College 3 – RNN

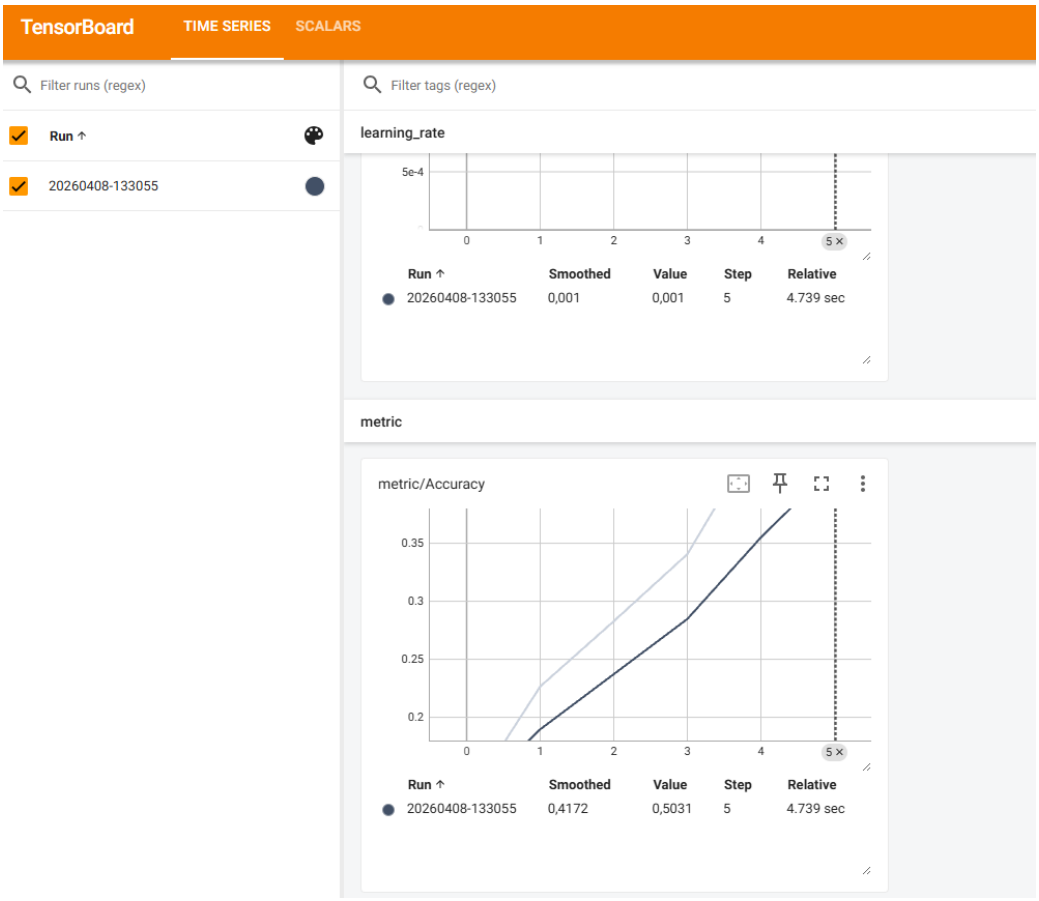
Same principle as previous portfolio exercises.

First, I start with the baseline run:

Figure 15 & 16

Baseline run, results in TensorBoard

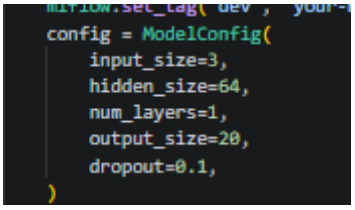




For the second iteration, I decided to tweak the parameters of the GRU model

Figure 17

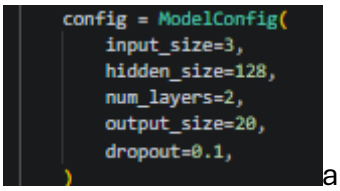
Overview of settings in VSCode script, original parameters



original parameters

Figure 18

Overview of settings in VSCode script, adjusted parameters

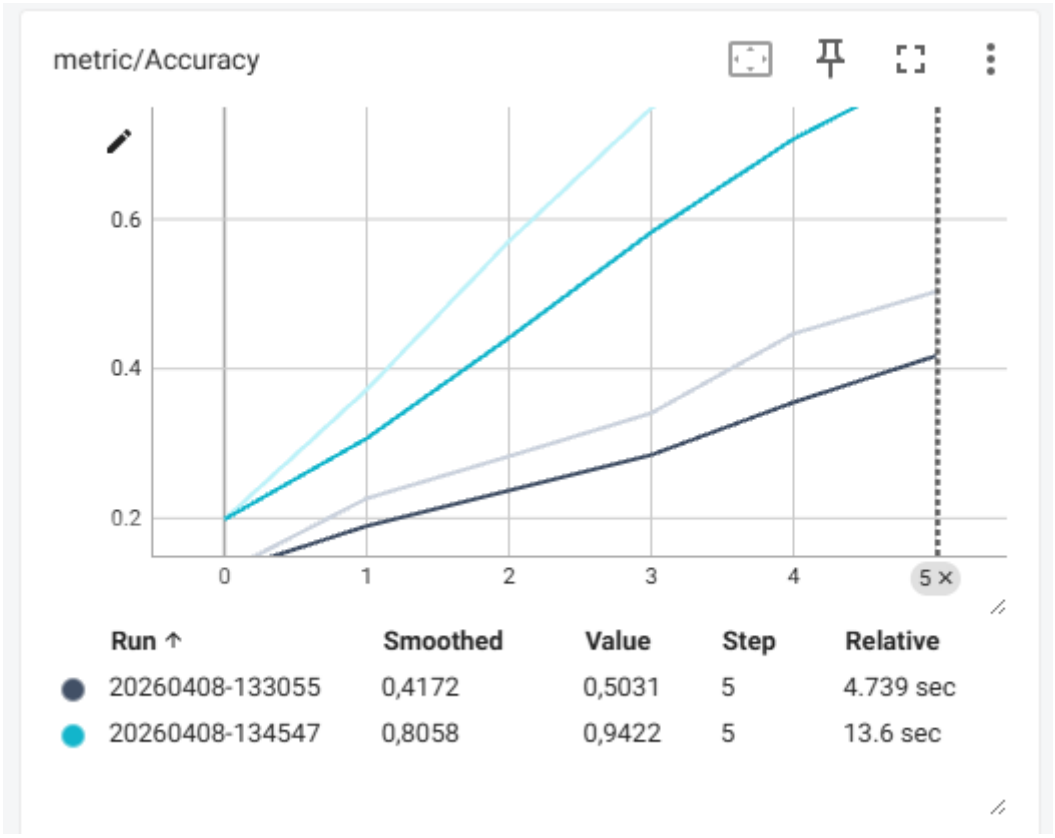


adjusted parameters

This resulted in longer training, but also in….:

Figure 19

Overview of accuracy score in MLFlow of latest improved run

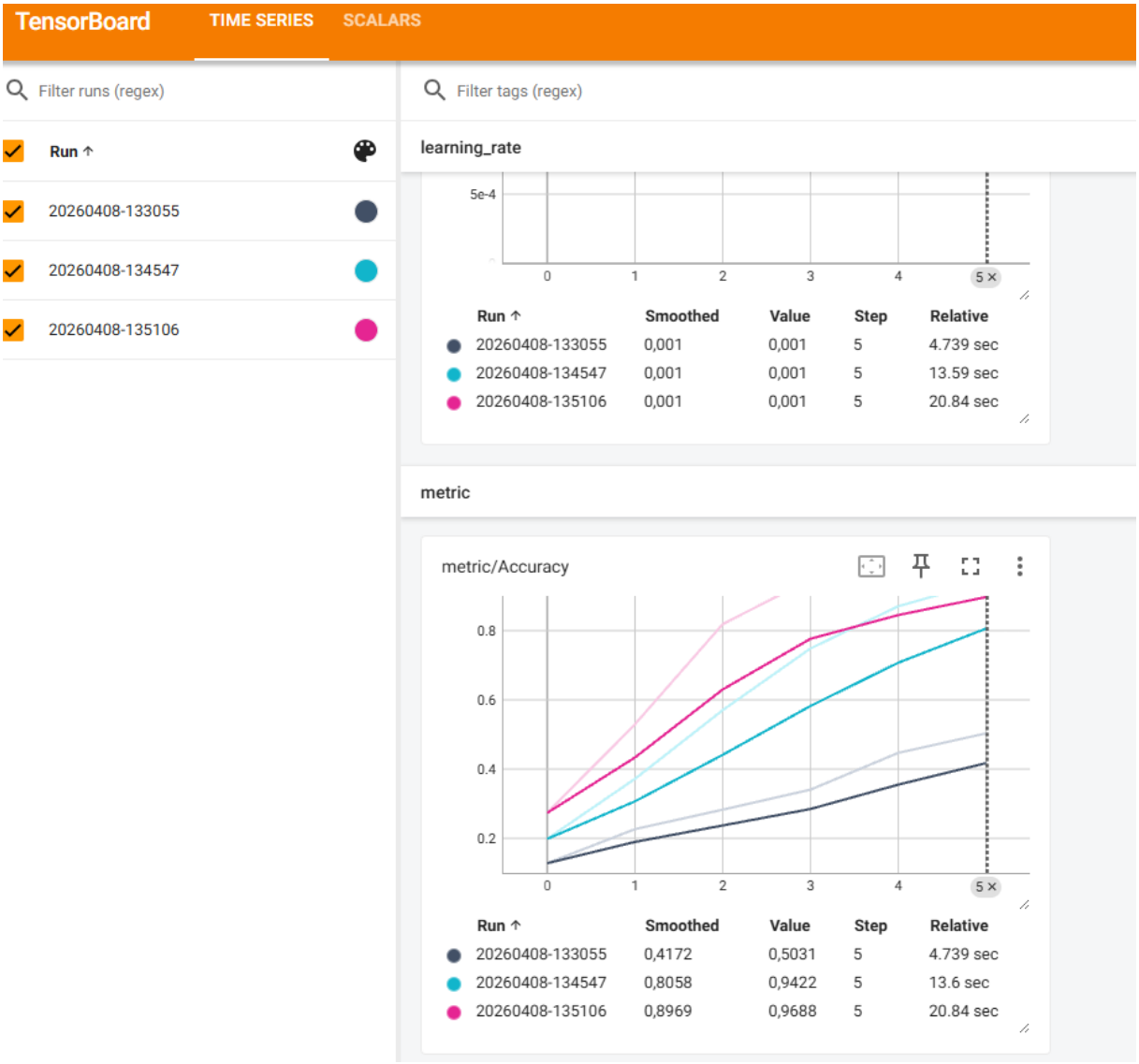


A way better accuracy score!

Next I increased hidden size even more and also dropout a bit to prevent overfitting.

Figure 20

Latest run, results in TensorBoard, increasing hidden size and adding dropout



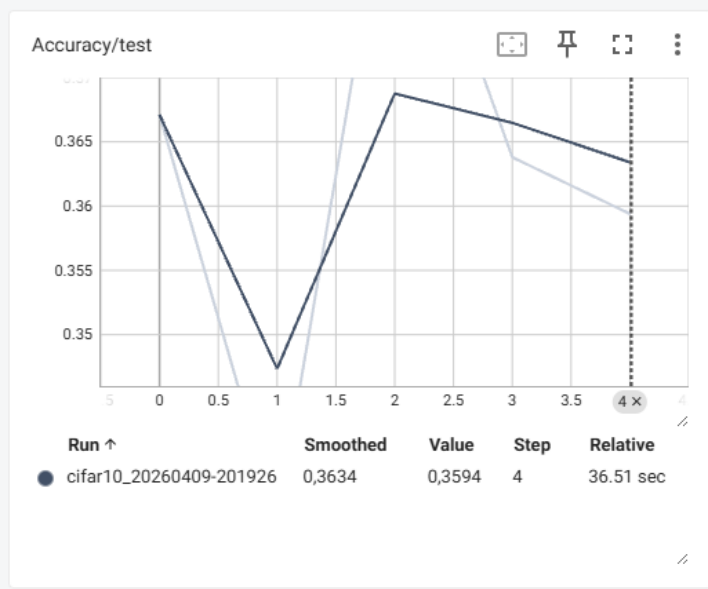
This last run achieved an accuracy well above the mentioned goal in the script, so it seems like mission successful.

Portfolio assignment – College 4 – Hypertuning

I wanted to challenge myself with a new dataset, so I chose the CIFAR(10) dataset. I've also added the script in my Github, so you can see how I apply my logic throughout the script. I started off by implementing a simple baseline model to ensure proper training, tuning and logging. Baseline model has poor accuracy (as would be expected), 0.3594* after 5 epochs. (*Note added later: I noticed in my later runs that my outcomes were not deterministic, so I implemented a fixed seed to ensure proper testing from there on out. This is why the value of the screenshot is different from my table below)

Figure 21

Overview results in TensorBoard, accuracy score baseline run



My hypothesis is: *Increasing the complexity of my model will improve classification accuracy on CIFAR-10, up to a point where performance flips due to overfitting.*

I've done various improvements, I've added a table below that shows the results per improvement.

Table 1

Hypertuning improvement iterations

Model	Improvement	Accuracy	Comments
Baseline	None, baseline model.	0.3829	Epochs = 5
Improved_v1	Conv2d model	0.3966	Still linear
Improved_v2	Conv2d non linear	0.5587	Added ReLU
Improved_v3	Raised hidden units to 32 (doubled)	0.5969	

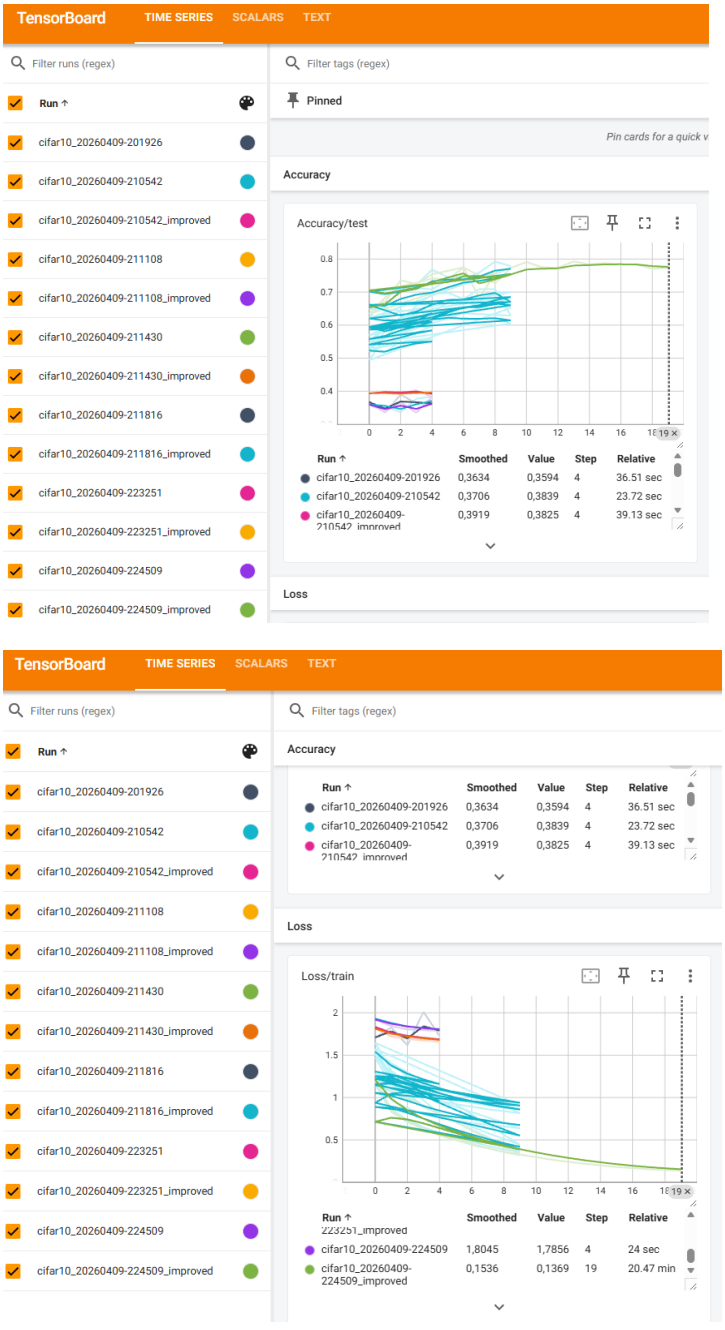
Improved_v4	Added a layer	0.6297	Learning curve still steep, so will increase epochs
Improved_v5	Increased epochs to 10	0.6049	Accuracy actually decreased, I will add pooling now
Improved_v6	Added pooling	0.6675	Epochs to 5 again for a bit faster training
Improved_v7	Epochs to 10 again to see if it helps…	0.6823	
Improved_v8	Added dropout	0.6525	
Improved_v9	Added another layer	0.6998	
Improved_v10	Added BatchNorm	0.6289	Steady increase until last epoch, then a massive dip from 0.7085 to 0.6289. Overall over the epochs, accuracy improved so improvement seem to have been useful. I will continue training for now with different improvements but will keep this in mind.
Improved_v11	Increased amount of channels	0.7743	Training seems stable now.
Improved_v12	Adding classifier head (before final classification)	0.7785	One run had 0.7924
Improved_v13	Added random horizontal flip to training	0.7745	
Improved_v14	Added random crop	0.7745	Other epochs had different accuracy, so seems like a coincidence that accuracy is the same
Improved_v15	Increased epochs to 20	0.7731	But training loss improved nice and

			steady. Model seems to be quite stable at this point.

The readme shows that there was a 18% test error. My model reached 77.31% accuracy, which I think comes close already to the mentioned test error with optimal hyperparameter tuning. Below I've added the curves of training as visualized via TensorBoard.

Figure 22 & 23

Overview results in TensorBoard, accuracy scores of runs during training and testing

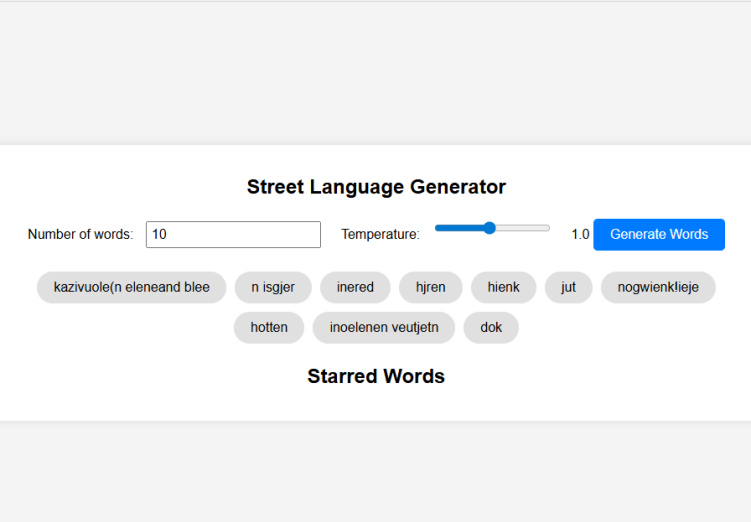


Portfolio assignment – College 5 – Deployment

For this assignment, I created a street language generator, deployed and accessible via my VM on SURF. See Figure below which shows the output of my 'Zeeuws' dialect generator.

Figure 24

Interactive version of Street Language Generator – Zeeuws dialect

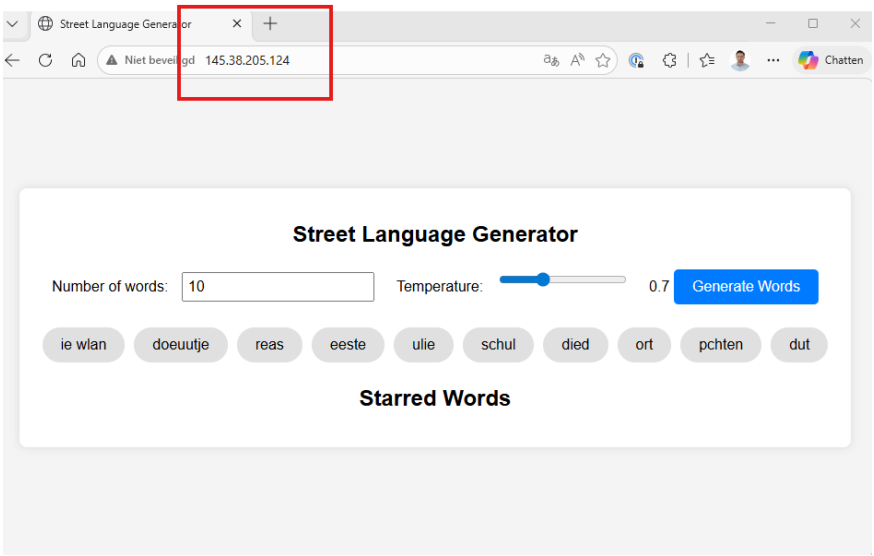


The generator is accessible via the URL http://145.38.205.124, after activating the VM and using the passphrase: 'dlmax'.

See Figure below which shows that the generator is available via the URL as shown above.

Figure 25

Interactive version of Street Language Generator – Zeeuws dialect, including HTTP



Portfolio assignment - Ethics part one

Evaluating ethical models & application on data ethics

Various ethical approaches provide a framework to resolve moral dilemmas, such as a 'Deontology' approach (Misselbrook, 2013). The deontology approach is mainly influenced by Immanual Kant and evaluates actions based on how they apply to universal actions, duties and principles, independent of the outcome. In other words, an action is ethical if it is morally correct. Virtue ethics offers a different approach. Mainly influenced by Aristotle, the focus lies on character and being a virtuous person (Van Hooft, 2014). The main thought behind being a virtuous person according to Aristotle, is to achieve 'Eudaimonia' which refers to the ultimate goal; to achieve human flourishing (Huta, 2013). An action is evaluated based on topics like integrity, responsibility, fairness and how it contributes to achieving eudaimonia. Therefore, an action is ethical if it is virtuous and contributes to achieving the ultimate goal.

On a finer granularity, ethical models provide a framework on how to handle moral cases, such as Impact Assessment Mensenrechten en Algoritmes (IAMA) and Koninklijke Nederlandsche Maatschappij tot bevordering der Geneeskunst (KNMG).

The IAMA framework (Ministerie van Binnenlandse Zaken en Koninkrijksrelaties, 2021) focuses on assessing the human rights impact of algorithmic systems before and during deployment. It evaluates aspects such as discrimination, transparency, and potential societal harms. The purpose of IAMA is to identify risks as early as possible and ensure that the developed algorithmic systems respect fundamental rights. An action is considered ethical when the algorithmic system can be justified in terms of human rights, transparency, and responsible governance.

The KNMG (KNMG, n.d.) ethical framework approaches moral dilemmas from a medical decision making perspective. The model outlines six clear steps on how to handle moral cases. The emphasis lies on careful moral reflection rather than on a purely technical risk assessment. The KNMG approach encourages users to examine a case from multiple perspectives, weigh conflicting values, and reach a justified ethical decision.

Both models provide a framework on how to handle a moral dilemma, but differ in their application and in their field of practice. The IAMA framework is a bit more bureaucratic (as it finds it origin in the government), while the KNMG model can be seen as a more practical approach (as it finds it origin in the medical field).

Suppose a hospital considers implementing an algorithm that predicts the remaining life expectancy of patients. Using the IAMA framework, the primary focus would be on evaluating risks such as bias in the training data, transparency of the model, possible discrimination, and the impact on fundamental rights. The goal would be to determine

whether the algorithm can be deployed responsibly and which safeguards are required (all in all quite bureaucratic without a real consideration for ethical implications).

Using the KNMG framework, the focus would lie more on the ethical implications, such as whether communicating model predictions to patients would be worth it (which is a practice that follows the main thought of virtue ethics). Medical professionals would have a discussion about questions such as whether providing this information benefits the patient, whether it could cause harm, and how it affects patient dignity.

The outcome of both approaches could lead to similar conclusions, but the reasoning behind the decision differs. In my view, while any situation is highly context dependent, the KNMG ethical approach is generally more useful for data ethics dilemmas. Ethical situations are often highly context dependent, and rule based approaches do not always capture the complexity of real world situations. This does not mean that frameworks such as IAMA (in a sense a deontology approach) are useless, but that the application of these frameworks is context dependent as well. For example, IAMA could be more useful for identifying specific detailed risks in vast algorithmic systems. Ultimately, ethical decision making more often benefits from combining a structured framework with context dependent moral issues in data ethics cases and therefore I prefer the KNMG approach.

Portfolio assignment - Ethics part two

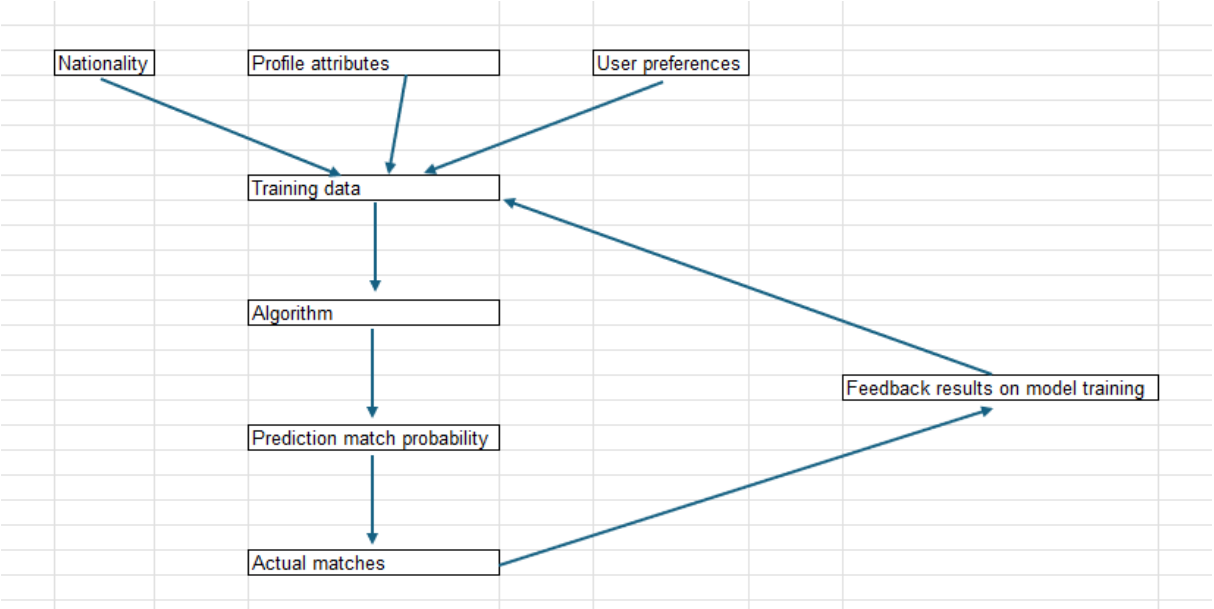
A Dutch dating app needs to adjust their algorithm to prevent discrimination. It was decided that the developers need to adjust their algorithm to make sure that non-Dutch users are being shown or matched as much as Dutch users (College voor de Rechten van de Mens, 2023).

My first impression is that the issue likely comes from the training data. Dating algorithms usually learn from past user behavior. If users historically liked or matched more often with certain groups, the model will learn that pattern. Lower visibility for non-Dutch users does not automatically mean the algorithm was designed to discriminate. It probably just means that the training data contained fewer samples of non-Dutch matches. Also, fewer matches with non-Dutch users can also be linked to user preferences of Dutch users such as profile quality, hobbies, language, or attractiveness. At the same time, there is a difference between individual preferences and fairness on a group level. For example, filtering by gender in a heterosexual dating app is functional, not discriminatory.

I doubt that forcing equal visibility is the best solution. It changes the results of the model instead of improving the model itself, and this may lower its performance. A better approach would be to first check whether certain groups are actually under represented for unfair reasons. This can be done by analyzing the data and letting humans review the results (semi supervised learning). If a real bias is found, specific adjustments can be made, instead of forcing full 'equality' for everyone (which may lower actual match percentages).

Figure 26

DAG – Dilemma algorithm dating app



I don’t see any direct difference with the first impression after drawing the DAG.

If a data scientist gets an assignment like this, the first step should be to understand how the system actually works before changing anything. There needs to be a check whether the problem is caused by the model itself or by actual user behavior/preferences. Then measure whether there is real unfairness in visibility or matching rates. If action is needed, choose a clear fairness goal and apply targeted adjustments instead of forcing equal results without analysis. Finally, document the tradeoff between fairness and model performance.

Portfolio – Summary Hackathon

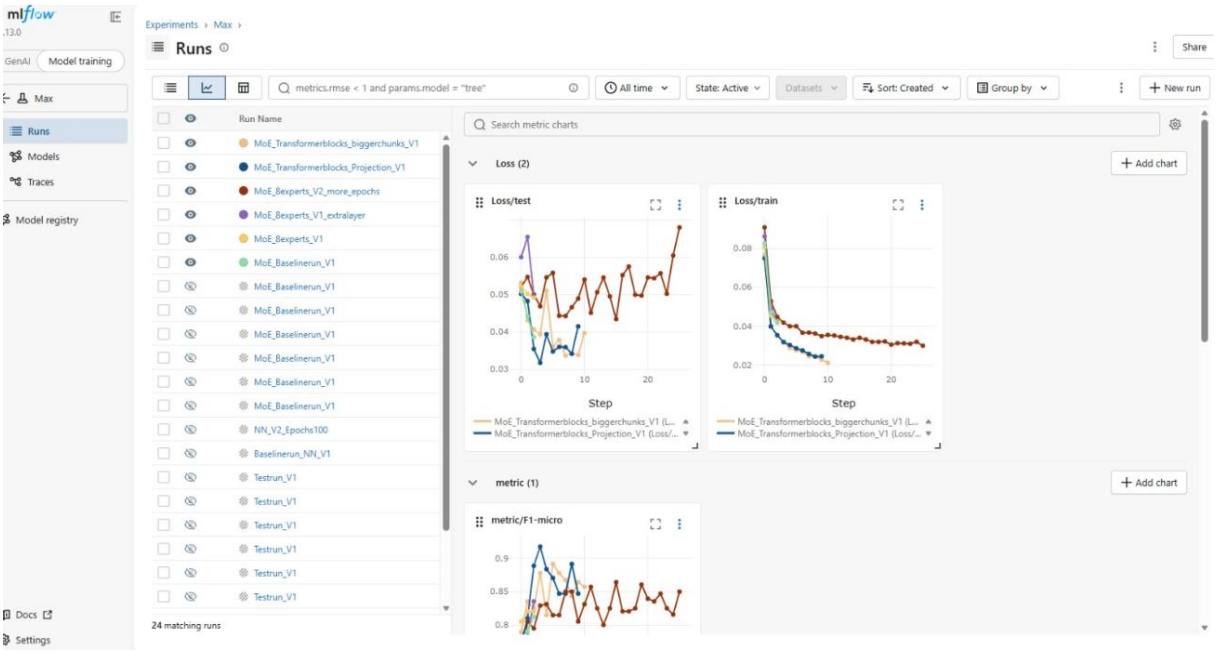
On 8 June 2026, I participated in the Deep Learning hackathon together with Jeffrey, Steven, and Alex as team Sigmoid Freud. The challenge was to classify approximately 20,000 legal deeds (aktes) into 32 possible classes, while also taking into account the fact that a deed could belong to multiple classes. One of the other main difficulties was the highly imbalanced dataset, where some of the smaller classes contained only contained 50 examples.

To help teams get started quickly, we received a preprocessed dataset from Raoul, allowing us to focus immediately on model development and experimentation.

Throughout the day, we trained and compared several deep learning techniques, including a simple neural network, a Transformer model, and a Mixture of Experts model. We also experimented with various preprocessing and data handling techniques, such as different chunk sizes and padding methods. All of our training, testing and tuning was logged via a group MLFlow dashboard. See an example of our logging in the figure below.

Figure 27

MLFlow logging during Hackathon - overview



The hackathon started at 09:45 and lasted until 17:00. Afterwards we had pizza and presented our results around 18:00. Overall an interesting day where we learned how to work with an actual dataset from a real company and where acquiring a good result has proven to be quite challenging.

References

College voor de Rechten van de Mens. (2023, 09 6). *Dating-app Breeze mag (en moet) algoritme aanpassen om discriminatie te voorkomen*. Opgehaald van mensenrechten: https://www.mensenrechten.nl/actueel/nieuws/2023/09/06/dating-app-breeze-mag-en-moet-algoritme-aanpassen-om-discriminatie-te-voorkomen

Huta, V. (2013). Eudaimonia. In *Oxford handbook of happiness* (pp. 201-213). Oxford University Press.

KNMG. (n.d.). *Ethische toolkit*. Opgehaald van KNMG: https://www.knmg.nl/ik-ben-geneeskundestudent-1/ethische-toolkit/zelf-aan-de-slag

Ministerie van Binnenlandse Zaken en Koninkrijksrelaties. (2021). *Impact Assessment Mensenrechten en Algoritmes (IAMA).* Opgehaald van https://www.uu.nl/sites/default/files/Rebo-IAMA.pdf

Misselbrook, D. (2013, 4). Duty, Kant, and deontology. *British Journal of General Practice, 63*(609), p. 211. Opgehaald van https://pmc.ncbi.nlm.nih.gov/articles/PMC3609464/

Van Hooft, S. (2014). *Understanding virtue ethics.* Routledge.